

Counting

Empty String

```
#include

using namespace std;
typedef long long ll;
const int maxN = 505;
const ll MOD = 1e9+7;

int N;
char S[maxN];
ll fac[maxN], inv[maxN], dp[maxN][maxN];

ll fastpow(ll x, ll b){
    ll res = 1;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

ll choose(int n, int k){
    if(k < 0 || n < k) return 0;
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;
}

void init(){
    fac[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fac[i] = (fac[i-1] * i) % MOD;
        inv[i] = fastpow(fac[i], MOD-2);
    }

    for(int i = 0; i < maxN; i++)
        for(int j = i; j < maxN; j++)
            dp[i][j] = -1;
}

ll solve(int l, int r){
    if((r-l+1)&1) return 0;
    if(l > r) return 1;
    if(dp[l][r] != -1) return dp[l][r];

    ll cnt = 0;
    for(int m = l+1; m <= r; m++){
        if(S[l] == S[m]){
            ll subcases = solve(l+1, m-1) * solve(m+1, r) % MOD;
            ll aftercombine = subcases * choose((r-l+1)/2, (m-l+1)/2) % MOD;
            cnt = (cnt + aftercombine) % MOD;
        }
    }

    return dp[l][r] = cnt;
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    init();
```

```
printf("%lld\n", N&1 ? 0 : solve(0, N-1));
}
```

Permutation Inversions

```
#include

using namespace std;
typedef long long ll;
const int maxN = 501;
const int maxK = maxN*(maxN-1)/2;
const ll MOD = 1e9+7;

int N, K;
ll dp[maxN][maxK];

void init(){
    for(int i = 1; i < maxN; i++){
        int r = i*(i-1)/2;
        dp[i][0] = dp[i][r] = 1;
        for(int j = 1; j <= r/2; j++){
            dp[i][j] = (dp[i-1][j] + dp[i][j-1]) % MOD;
            if(j-i >= 0)
                dp[i][j] = (dp[i][j] - dp[i-1][j-i] + MOD) % MOD;
        }
        for(int j = r/2+1; j < r; j++)
            dp[i][j] = dp[i][r-j];
    }
}

int main(){
    init();
    scanf("%d %d", &N, &K);
    printf("%lld\n", dp[N][K]);
}
```

Counting Bishops

```
#include

using namespace std;
typedef long long ll;
const int maxN = 501;
const ll MOD = 1e9+7;

int N, K;
ll fac[maxN], inv[maxN];
ll T[maxN], B[2*maxN], W[2*maxN];

ll fastpow(ll x, ll b){
    ll res = 1;
    while(b){

```

```

        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

ll choose(int n, int k){
    if(k < 0 || n < k) return 0;
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;
}

ll stirling2(int n, int k){
    ll sum = 0;
    for(int i = 0; i <= k; i++){
        ll a = (i&1 ? -1 : 1);
        ll b = choose(k, i);
        ll c = fastpow(k-i, n);
        ll term = a * b % MOD * c % MOD;
        sum = (sum + term + MOD) % MOD;
    }
    return sum * inv[k] % MOD;
}

void compute_T(int n){
    for(int k = 0; k <= n; k++){
        T[k] = stirling2(n+1, n+1-k);
    }
}

void init_choose(){
    fac[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fac[i] = (fac[i-1] * i) % MOD;
        inv[i] = fastpow(fac[i], MOD-2);
    }
}

ll calculate(int n, int k){
    if(n == 1) return 1;
    if(k >= 2*n) return 0;
    memset(B, 0, sizeof(B));
    memset(W, 0, sizeof(W));

    int m = n/2, odd = n&1;
    for(int k = 0; k <= m+odd; k++){
        compute_T(2*m-1-k+odd);
        ll a = choose(m+odd, k);
        ll b = choose(m, k);
        for(int i = 0; i+k < n; i++){
            B[i+k] += (a * T[i]) % MOD;
            B[i+k] %= MOD;
            W[i+k] += (b * T[i]) % MOD;
            W[i+k] %= MOD;
        }
    }

    ll sum = 0;
    for(int b = 0; b <= min(N-1, K); b++){
        int w = K-b;
        sum += (B[b] * W[w]) % MOD;
        sum %= MOD;
    }
    return sum;
}

int main(){
    init_choose();
    scanf("%d %d", &N, &K);

```

```
    printf("%lld\n", calculate(N, K));  
}
```

Counting Sequences

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 1e6+1;  
const ll MOD = 1e9+7;  
  
int N, K;  
ll fac[maxN], inv[maxN];  
  
ll fastpow(ll x, ll b){  
    ll res = 1;  
    while(b){  
        if(b&1) res = (res * x) % MOD;  
        x = (x * x) % MOD;  
        b >>= 1;  
    }  
    return res;  
}  
  
void init_choose(){  
    fac[0] = inv[0] = 1;  
    for(int i = 1; i < maxN; i++){  
        fac[i] = (fac[i-1] * i) % MOD;  
        inv[i] = fastpow(fac[i], MOD-2);  
    }  
}  
  
ll choose(int n, int k){  
    if(k < 0 || k > n) return 0;  
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;  
}  
  
ll T(int n, int k){  
    ll sum = 0;  
    for(int j = 0; j <= k; j++){  
        ll a = (j&1 ? -1 : 1);  
        ll b = choose(k, j);  
        ll c = fastpow(k-j, n);  
        ll term = (a * b % MOD * c % MOD + MOD) % MOD;  
        sum = (sum + term) % MOD;  
    }  
    return sum;  
}  
  
int main(){  
    init_choose();  
    scanf("%d %d", &N, &K);  
    printf("%lld\n", T(N, K));  
}
```

Grid Paths II

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e6+1;
const int maxM = 1002;
const ll MOD = 1e9+7;

int N, M, x, y;
ll fact[maxN], inv[maxN], dp[maxM];
struct point {int x, y;} p[maxM];

ll inverse(ll a){
    ll res = 1;
    ll b = MOD-2;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

ll choose(int n, int k){
    return fact[n] * inv[n-k] % MOD * inv[k] % MOD;
}

int main(){
    init();
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &x, &y);
        p[i] = {x, y};
    }
    p[M] = {N, N};
    sort(p, p+M+1, [](point a, point b){
        return a.x == b.x ? a.y < b.y : a.x < b.x;
    });

    for(int i = 0; i <= M; i++){
        dp[i] = choose(p[i].x+p[i].y-2, p[i].x-1);
        ll subtract = 0;
        for(int j = 0; j < i; j++){
            if(p[i].x >= p[j].x && p[i].y >= p[j].y){
                int dx = p[i].x-p[j].x, dy = p[i].y-p[j].y;
                subtract += dp[j] * choose(dx+dy, dx);
                subtract %= MOD;
            }
        }
        dp[i] = (dp[i] - subtract + MOD) % MOD;
    }
    printf("%lld\n", dp[M]);
}
```

Counting Permutations

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1000;
const ll MOD = 1e9+7;

int N;
ll dp[maxN+1];

void init(){
    dp[0] = dp[1] = 1;
    dp[2] = dp[3] = 0;
    for(int i = 4; i <= maxN; i++){
        ll a = (i+1) * dp[i-1] % MOD;
        ll b = (i-2) * dp[i-2] % MOD;
        ll c = (i-5) * dp[i-3] % MOD;
        ll d = (i-3) * dp[i-4] % MOD;

        dp[i] = (a-b-c+d);
        while(dp[i] < 0)
            dp[i] += MOD;
        dp[i] %= MOD;
    }
}

int main(){
    init();
    scanf("%d", &N);
    printf("%lld\n", dp[N]);
}
```

Grid Completion

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;
const int maxN = 505;

int N, p[maxN], q[maxN], C[6];
bool inp[maxN], inq[maxN];
char S[maxN];
ll fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
}
```

```

    }
    return res;
}

ll choose(int x, int y){
    return (fact[x] * inv[y] % MOD) * inv[x-y] % MOD;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i <= N; i++){
        fact[i] = (fact[i-1] * i) % MOD;
        inv[i] = (inv[i-1] * inverse(i)) % MOD;
    }
}

ll f(int i, int j, int k){
    ll res = (choose(C[0], i)
        * choose(C[1], j) % MOD
        * choose(C[2], k) % MOD
        * choose(C[3], i) % MOD);

    res = (res * fact[i] % MOD
        * fact[C[4]-i-j] % MOD
        * fact[C[5]-i-k] % MOD);

    if((i+j+k) % 2 == 1)
        res = (MOD - res);

    return res;
}

int main(){
    scanf("%d", &N);
    init();
    for(int i = 0; i < N; i++){
        scanf(" %s", S);
        p[i] = q[i] = -1;
        for(int j = 0; j < N; j++){
            if(S[j] == 'A') { p[i] = j; inp[j] = true; }
            if(S[j] == 'B') { q[i] = j; inq[j] = true; }
        }
    }

    for(int i = 0; i < N; i++){
        if(p[i] == -1 && q[i] == -1) C[0]++;
        if(p[i] == -1 && q[i] != -1 && !inp[q[i]]) C[1]++;
        if(p[i] != -1 && q[i] == -1 && !inq[p[i]]) C[2]++;
    }

    for(int i = 0; i < N; i++){
        if(!inp[i] && !inq[i]) C[3]++;
        if(!inp[i]) C[4]++;
        if(!inq[i]) C[5]++;
    }

    ll ans = 0;
    for(int i = 0; i <= min(C[0], C[3]); i++)
        for(int j = 0; j <= C[1]; j++)
            for(int k = 0; k <= C[2]; k++)
                ans = (ans + f(i, j, k)) % MOD;
    printf("%lld\n", ans);
}

```

Functional Graph Distribution

```
#include

using namespace std;
typedef long long ll;
const int maxN = 5001;
const ll MOD = 1e9+7;

int N;
ll pown[maxN], fac[maxN], inv[maxN], S[maxN][maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

void init_powers(){
    pown[0] = 1;
    for(int i = 1; i < maxN; i++){
        pown[i] = (pown[i-1] * N) % MOD;
    }
}

void init_choose(){
    fac[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fac[i] = (fac[i-1] * i) % MOD;
        inv[i] = inverse(fac[i]);
    }
}

void init_stirling(){
    S[1][1] = 1;
    for(int n = 2; n < maxN; n++){
        for(int k = 1; k <= n; k++){
            S[n][k] = (S[n-1][k-1] - (n-1) * S[n-1][k]) % MOD;
        }
    }
}

ll choose(int n, int k){
    if(k < 0 || k > n) return 0;
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;
}

ll stirling1(int n, int k){
    return abs(S[n][k]);
}

ll T(int n, int k){
    ll sum = 0;
    for(int j = 0; j <= n-1; j++){
        ll a = choose(n-1, j);
        ll b = pown[n-1-j];
        ll c = stirling1(j+1, k);
        sum += a * b % MOD * c % MOD;
        sum %= MOD;
    }
    return sum;
}

int main(){
    scanf("%d", &N);
    init_powers();
```

```
init_choose();  
init_stirling();  
for(int k = 1; k <= N; k++)  
    printf("%lld\n", T(N, k));  
}
```
